



Inspiring Excellence

Network Layer

Routing Algorithm

Distance Vector Routing

Lecture 12 | CSE421 – Computer Networks

Department of Computer Science and Engineering

School of Data & Science

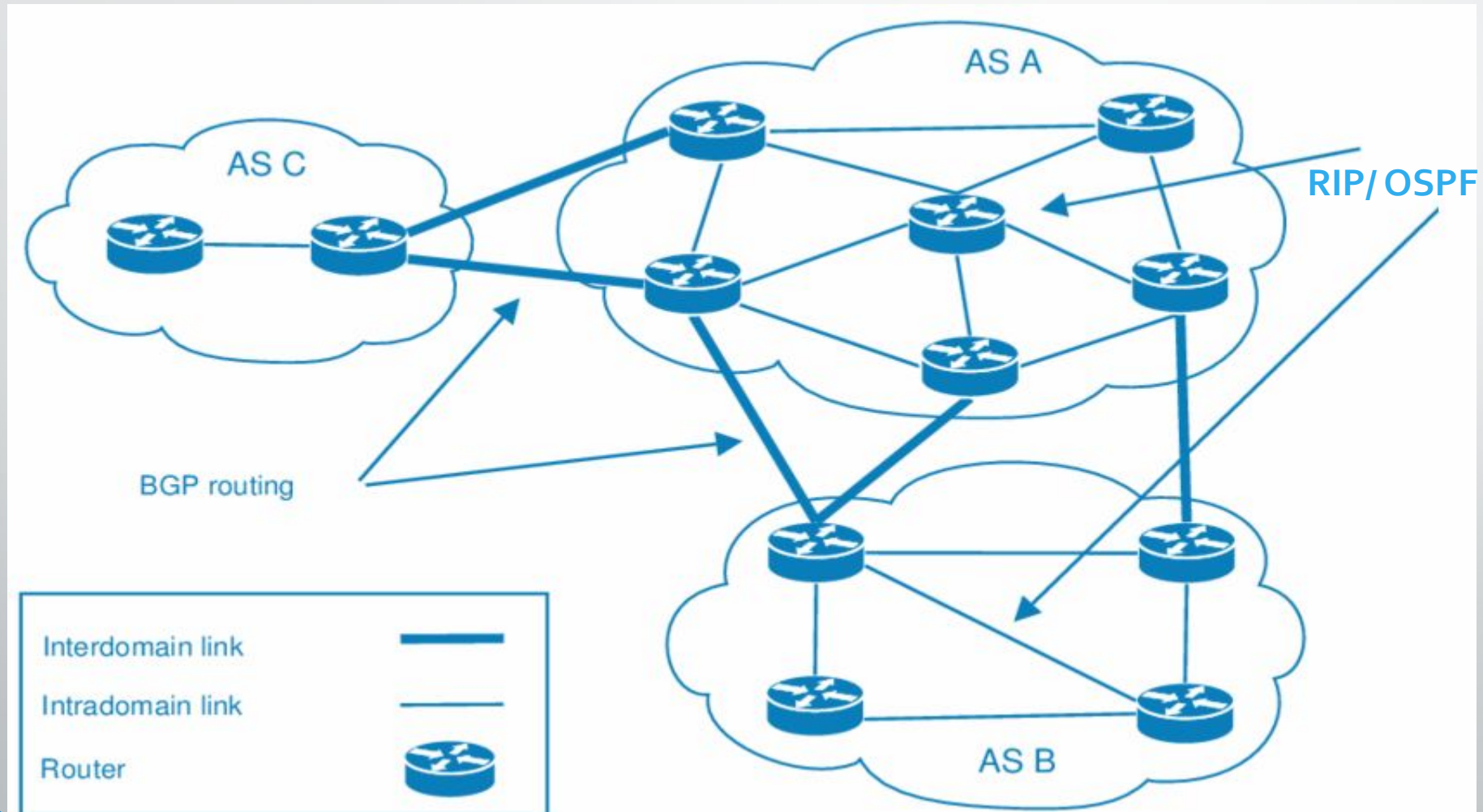
Objectives

- understand principles behind network layer services:
 - routing algorithms
 - **distance vector**
 - **link state**

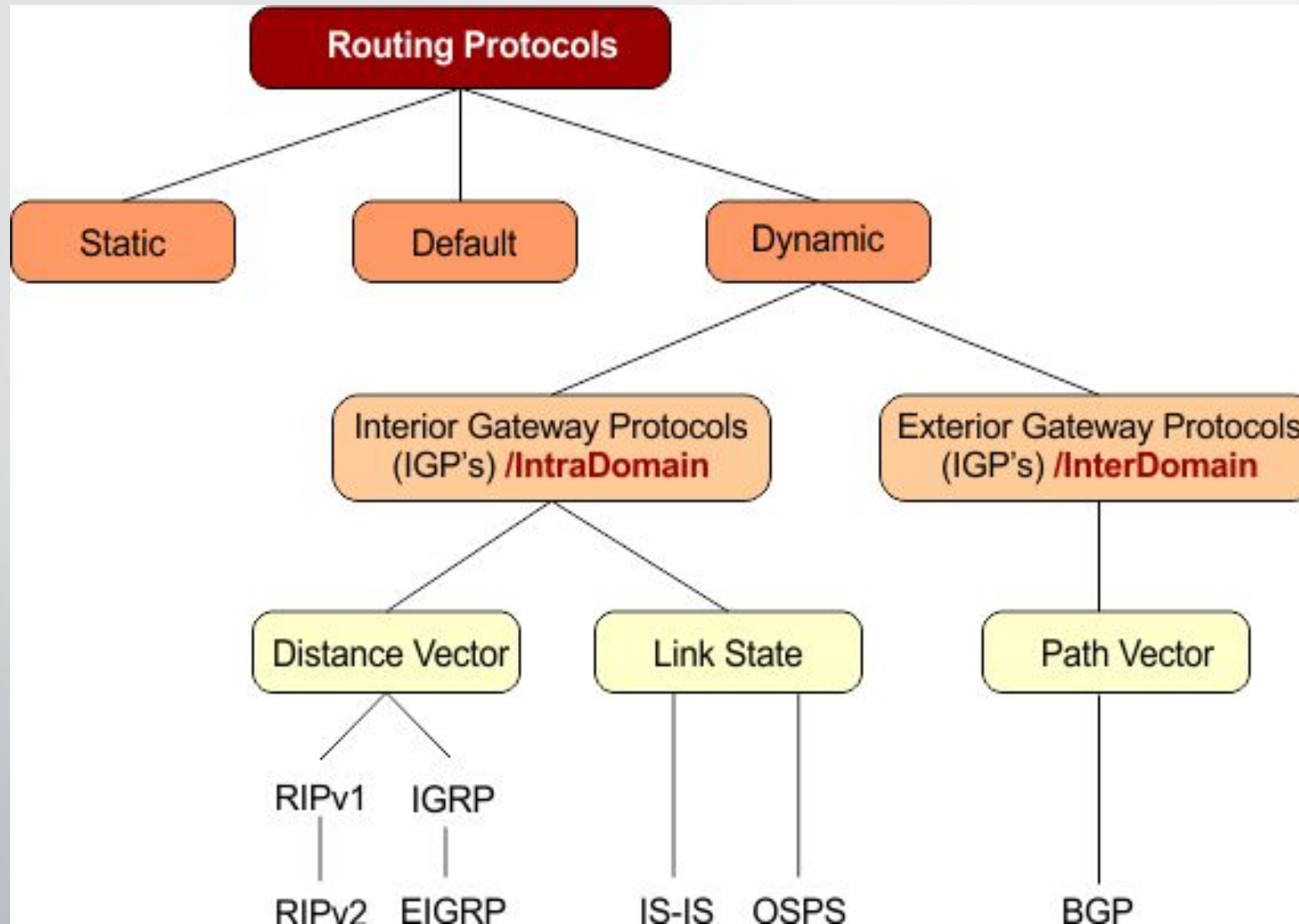
Autonomous Systems

- Internet is divided into autonomous systems (AS).
- An AS = a group of networks and routers controlled by one organization.
- Routing *inside* an autonomous systems(AS) : **intra-domain routing**.
- Routing *between* autonomous systems(AS) : **inter-domain routing**.

Autonomous Systems

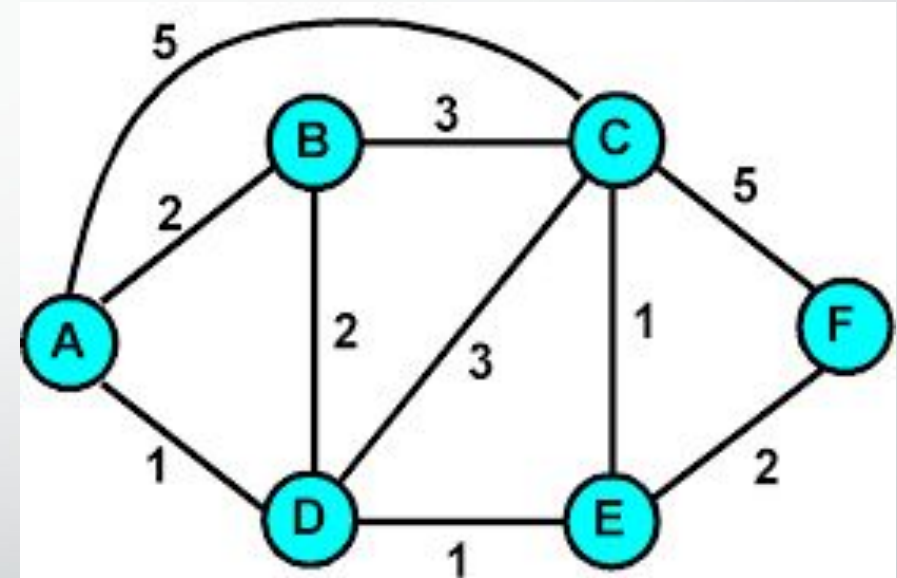


Popular Routing Protocols



Routing Algorithms

- Given a set of routers and links connecting the routers.
- Routing algorithm finds a “good” path from the source to destination router.
- Good path = Lowest cost path



Routing Algorithm classification

Global:

- Every router knows the whole network (all nodes, all links, and their costs).
- Each router builds a complete map (topology).
- Uses **link state algorithms** (like Dijkstra's shortest path).

Decentralized:

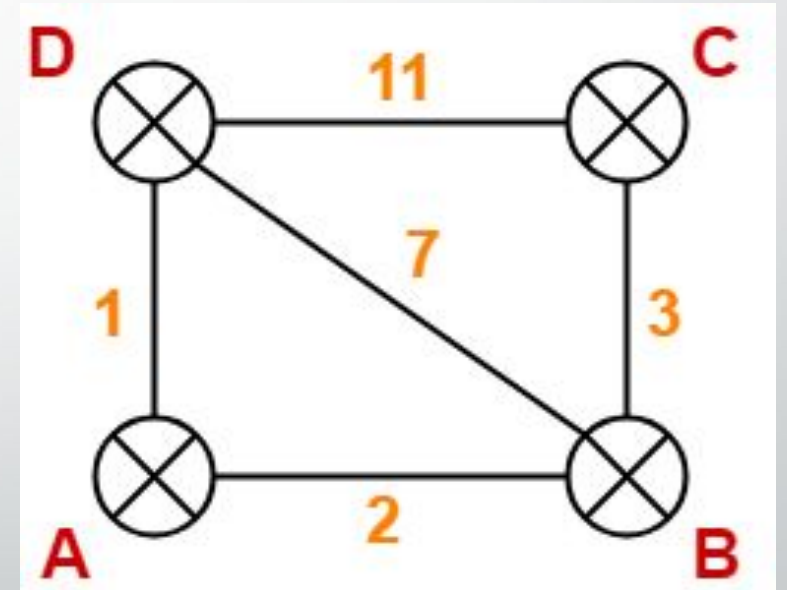
- Each router only knows about its neighbors and the cost to reach them.
- Routers share information with neighbors and update step by step.
- Uses **distance vector algorithms** (like Bellman-Ford).



Distance Vector Algorithm

Distance vector algorithm

- Based on **Bellman-Ford Algorithm**.
- Each router finds **shortest paths** to all destinations.
- Shortest path cost can be:
 - Hop count
 - Link delay
 - Bandwidth



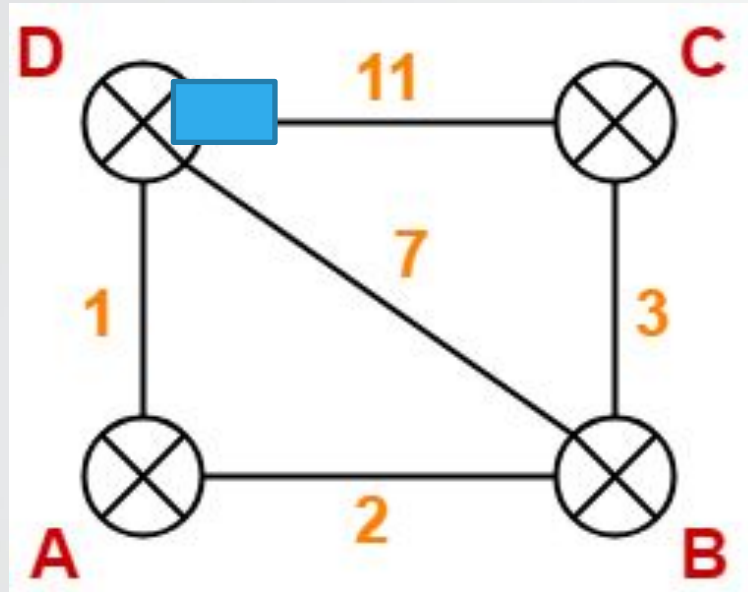
Uses a **weighted network (topology)**.

How Does It Work?

- Each router keeps a **Distance Vector (table)**.
- Router **shares its table** with neighbors regularly.
- When a router gets a table from a neighbor, it **updates its own table** using the Bellman-Ford rule.
- Repeat until all routers know the shortest paths.

NODE D Table

Node	Distance
A	1
B	7
C	11
D	0



NODE C Table

Node	Distance
A	12
B	3
C	0
D	11

Cost to A = cost(to neighbor) + neighbor's cost(to destination)

D Router		Cost to A from D Router
11	+	1

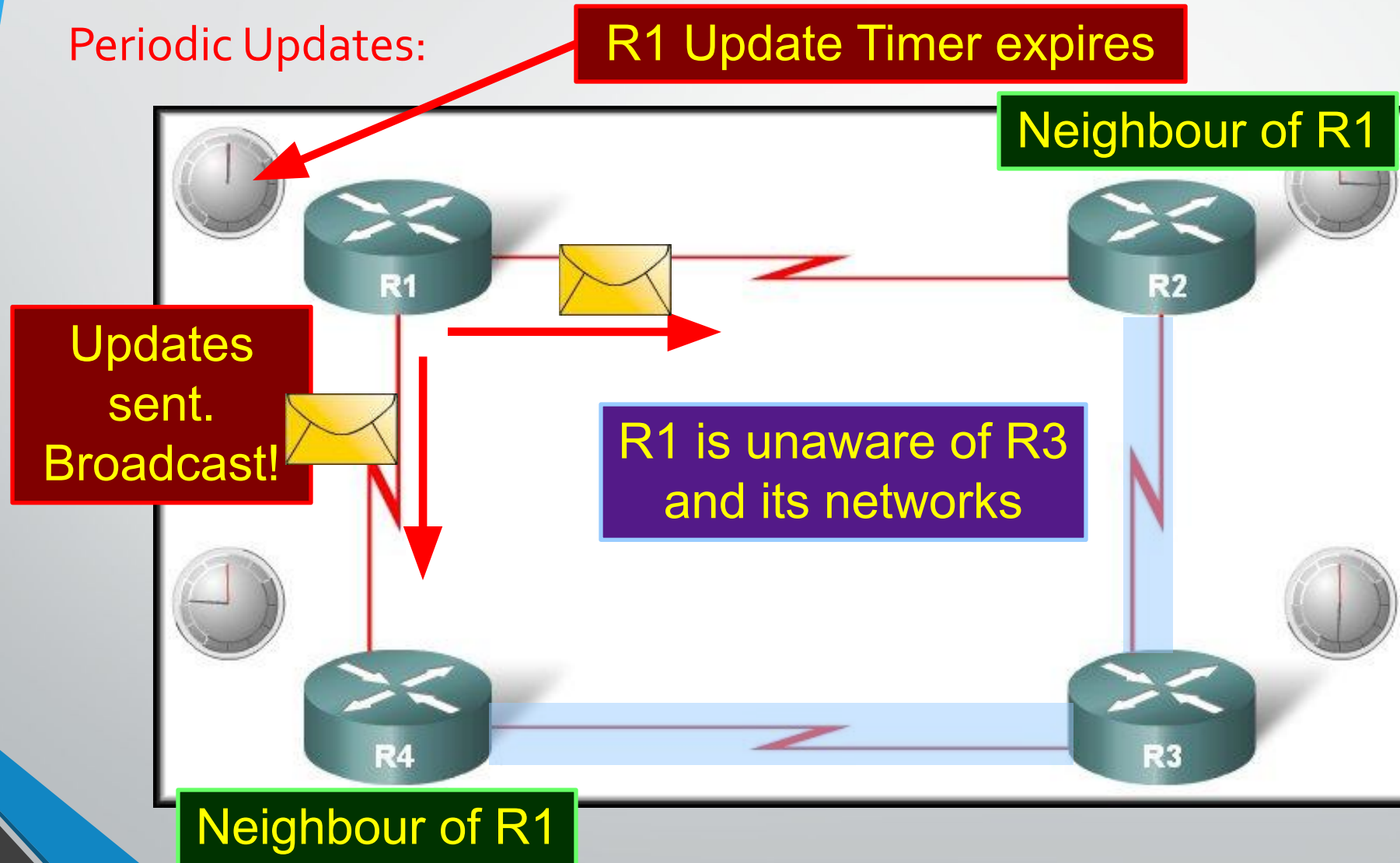
Distance Vector Algorithm:

Cost to a destination == cost(to neighbor) + neighbor's cost(to destination)

Periodic Updates of Distance Vector

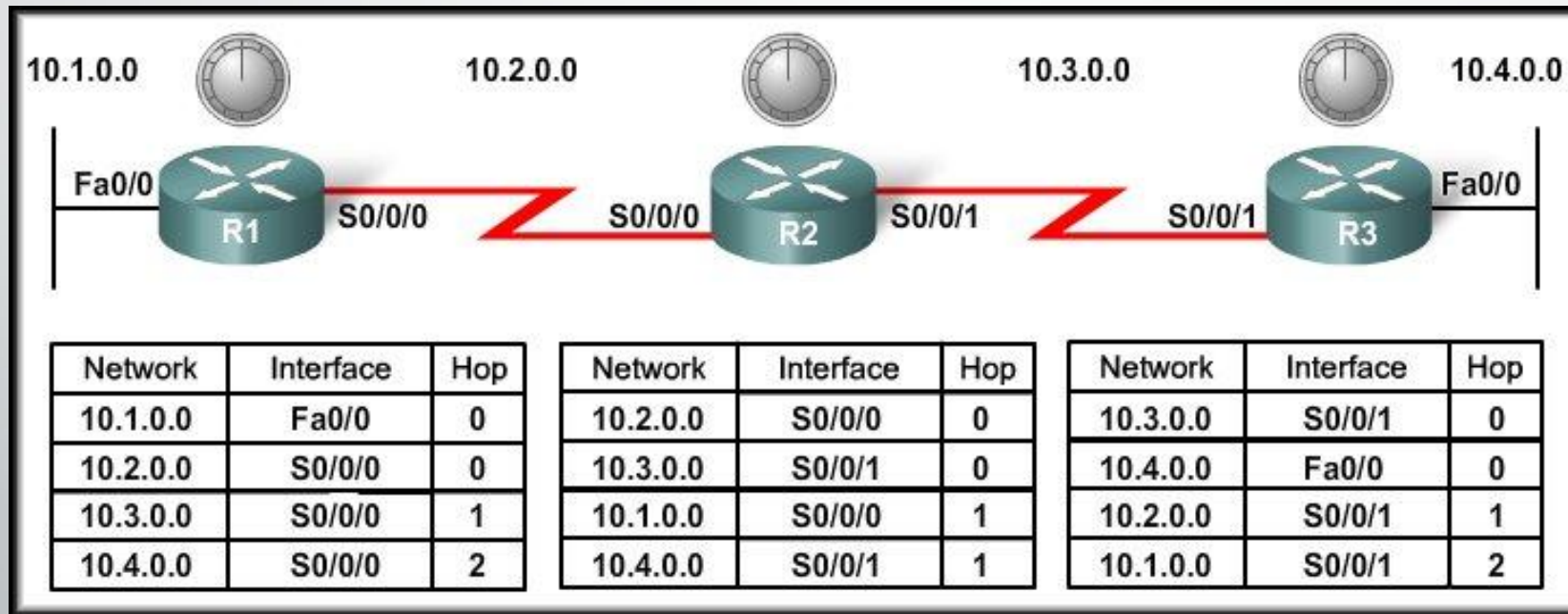
- Routers regularly send their **entire routing table** to neighbors (e.g., every 30 seconds in RIP).
- These updates are sent even when there **is no change** in the network.
- This is **simple but inefficient**.

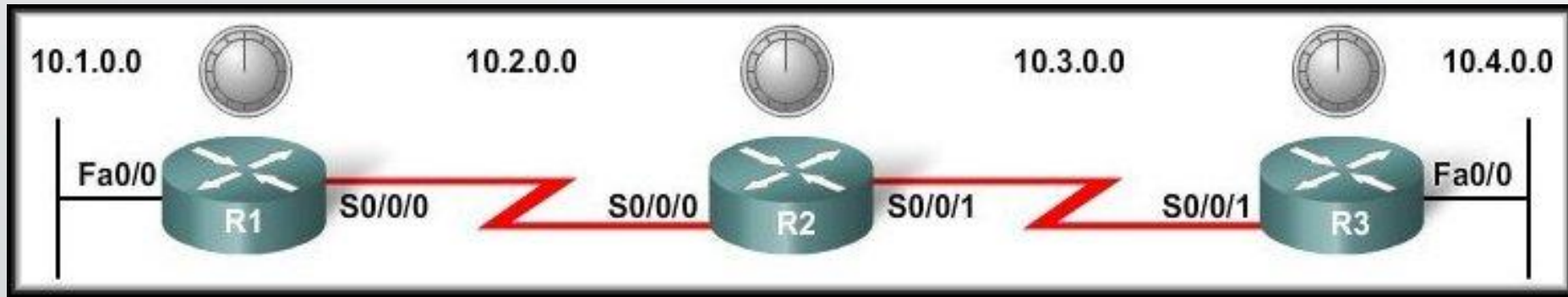
Operation of Distance Vector



Distance Vector Routing Protocols

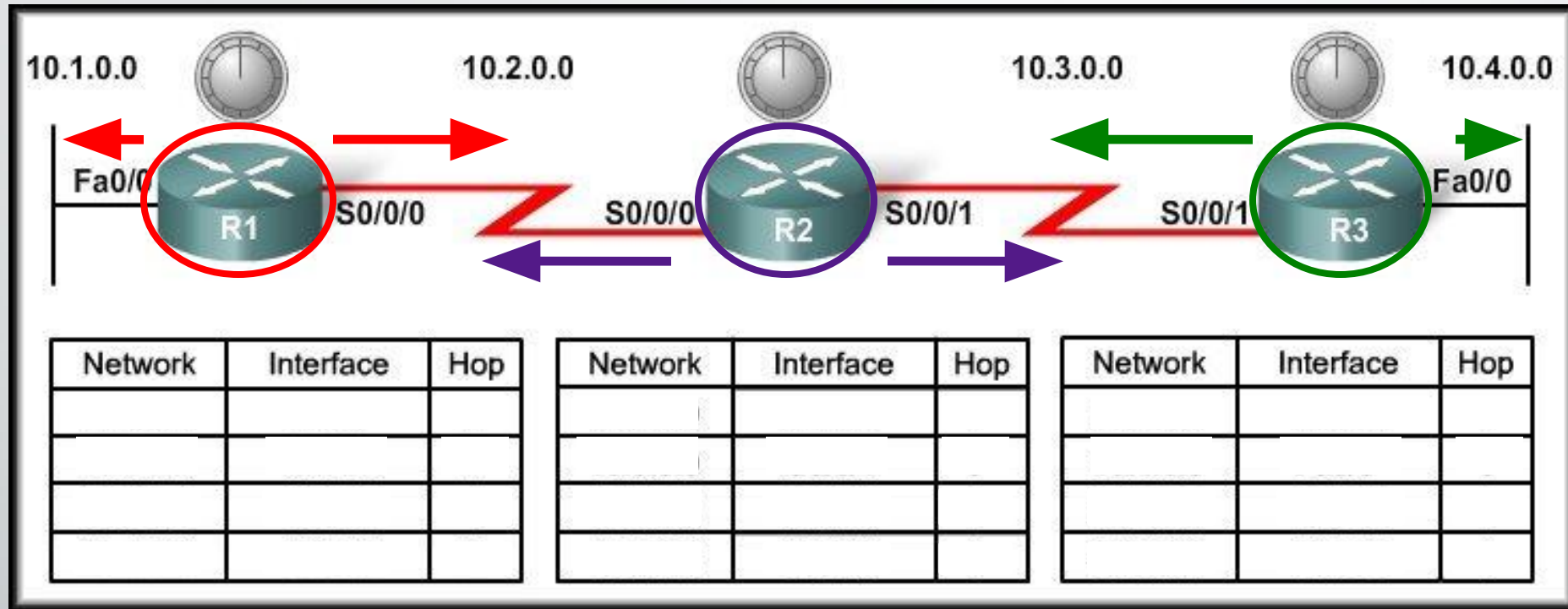
Network Discovery





- **Network Discovery:**
 - Is part of the process of the routing protocol algorithm that enables routers to **learn about remote networks for the first time.**

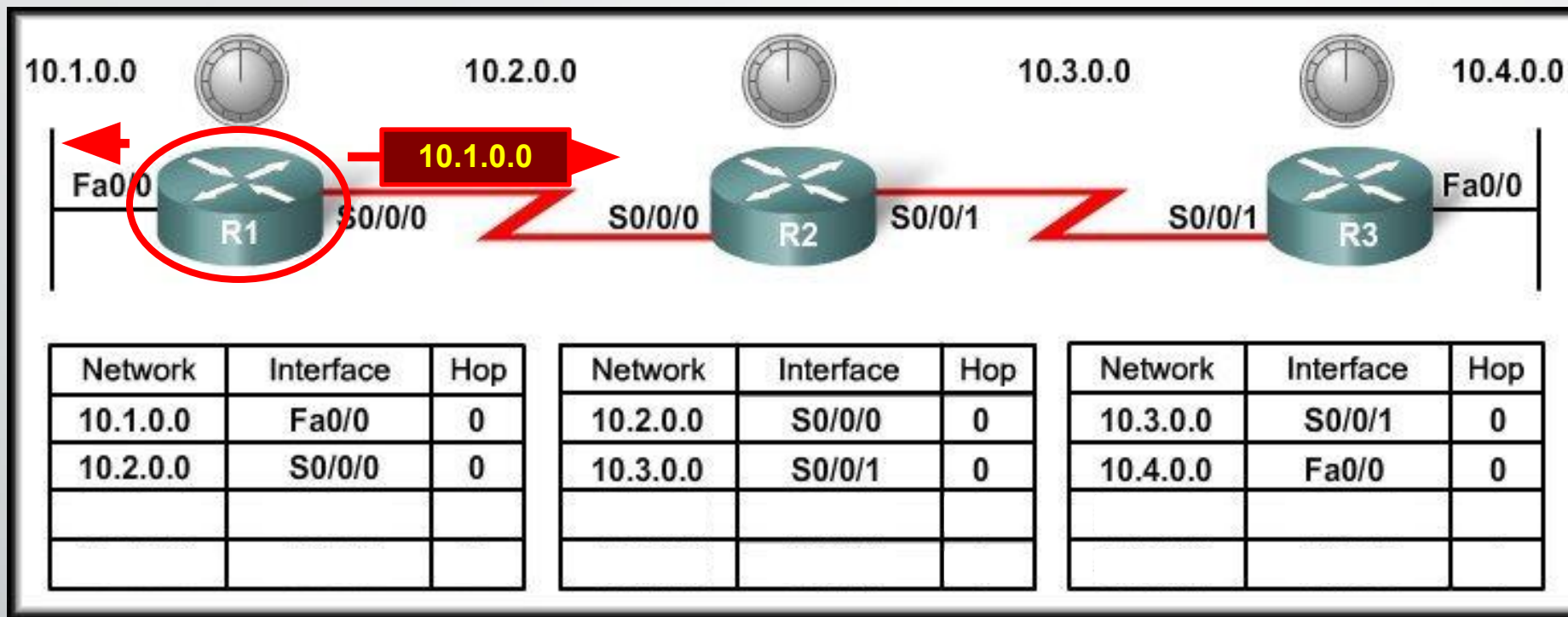
Cold Start



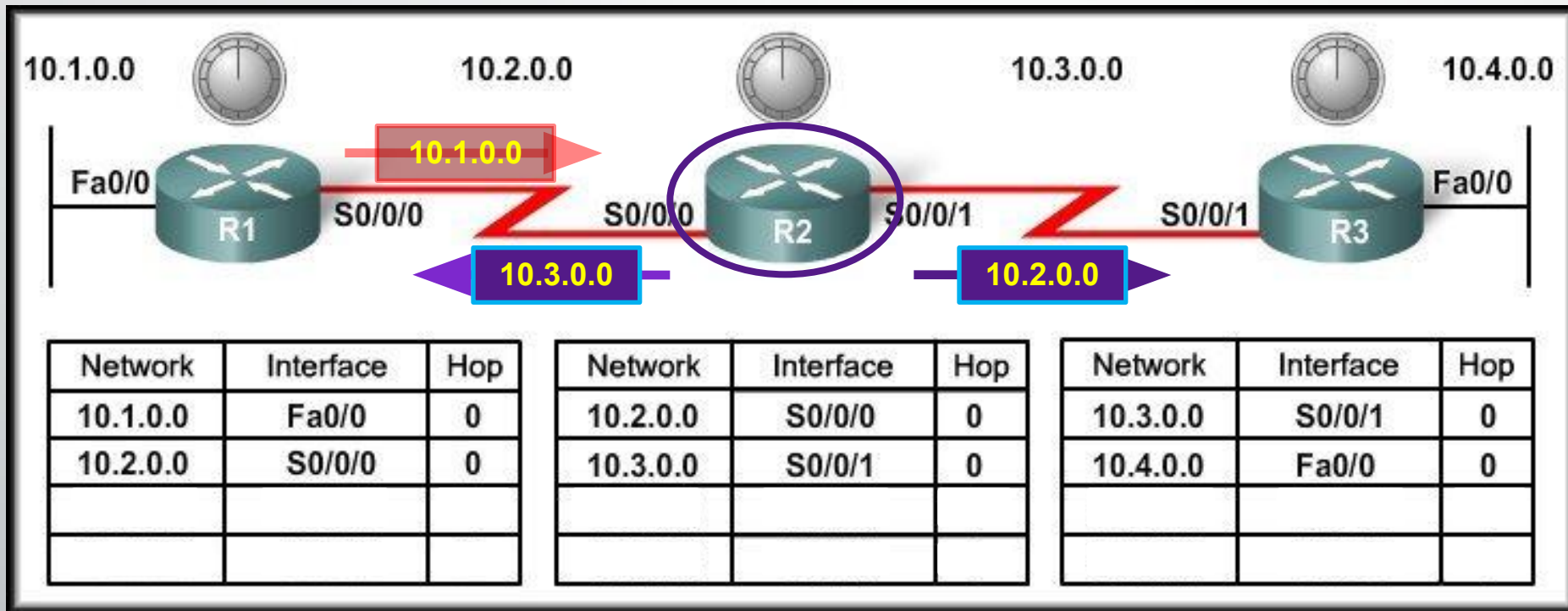
- When a router powers up:

- Knows nothing about the network topology.
- Knows only the information saved in NVRAM.
- Sends updates about its known networks out all ports.

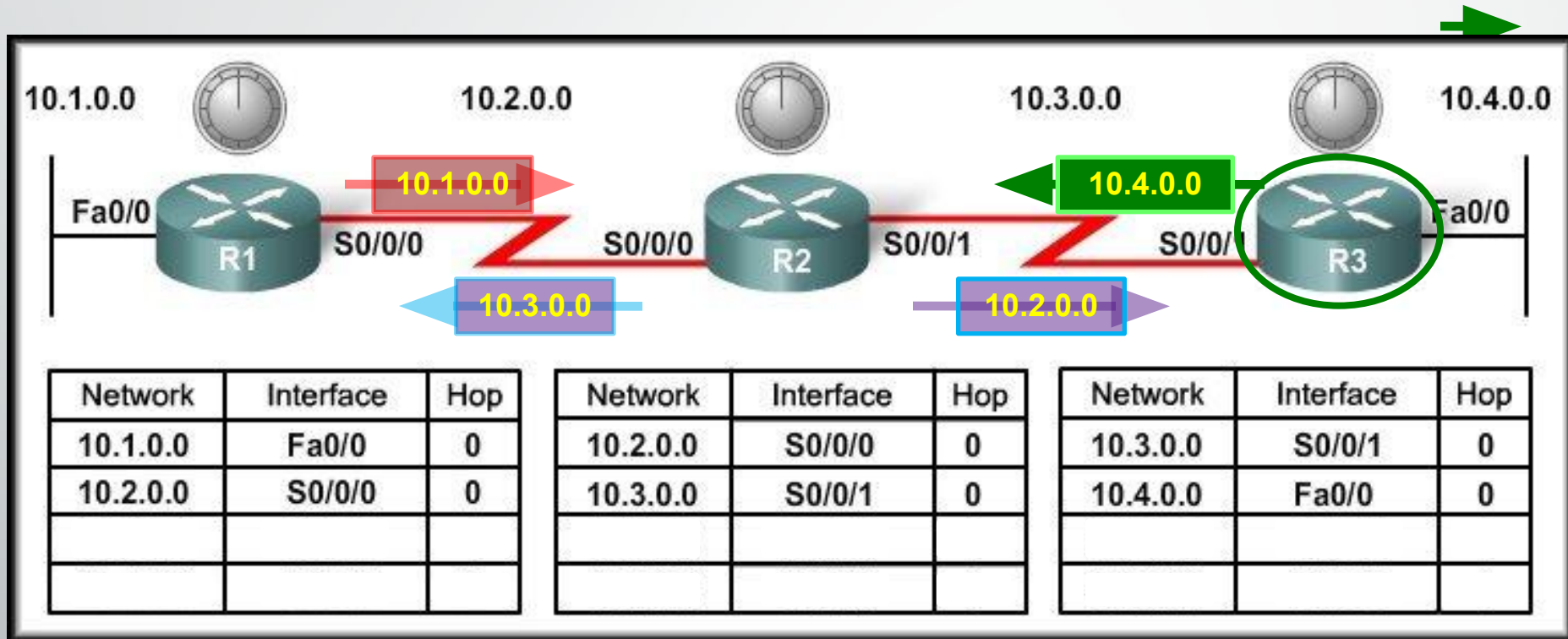
Initial Exchange of Routing Information



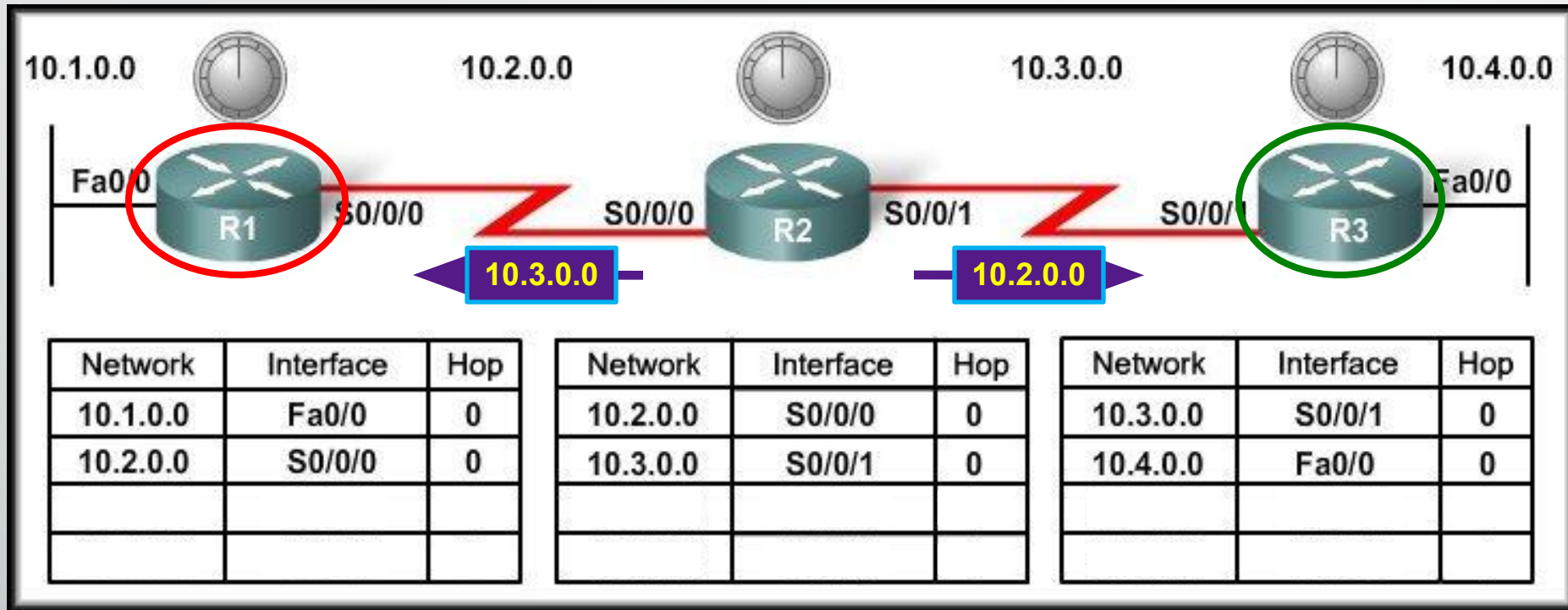
Initial Exchange of Routing Information



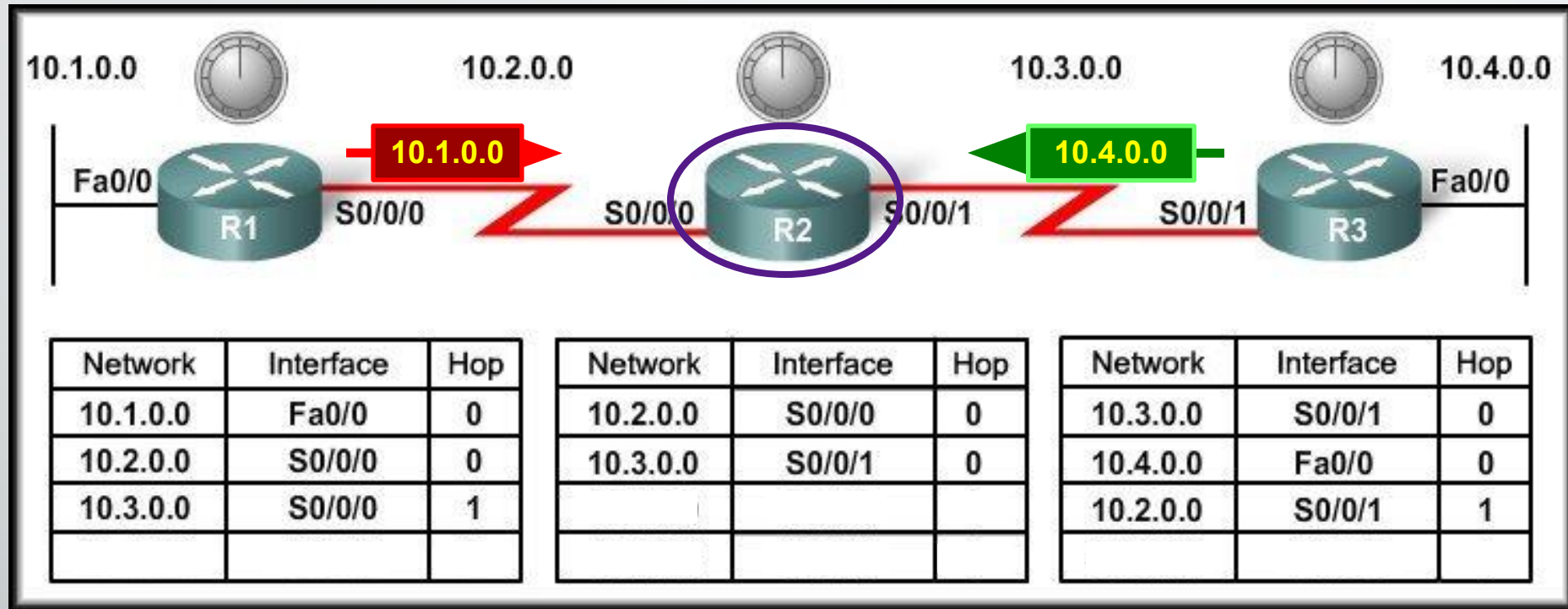
Initial Exchange of Routing Information



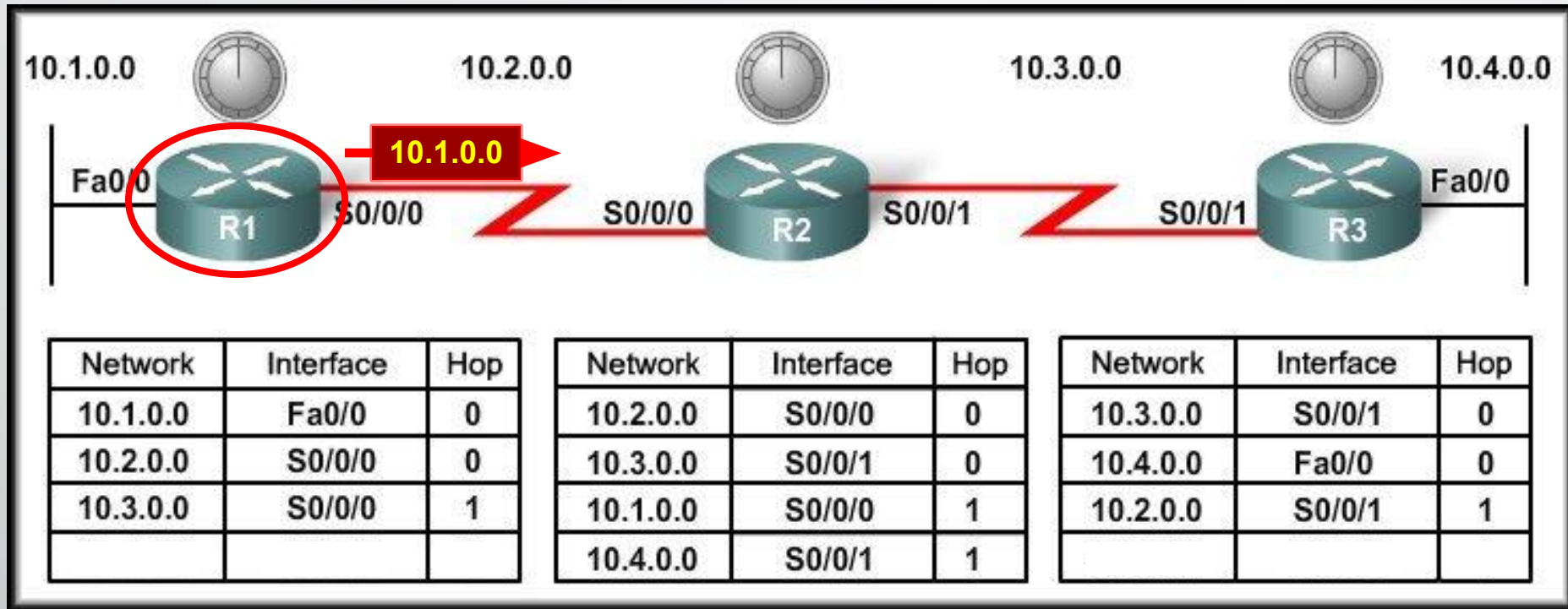
Initial Exchange of Routing Information



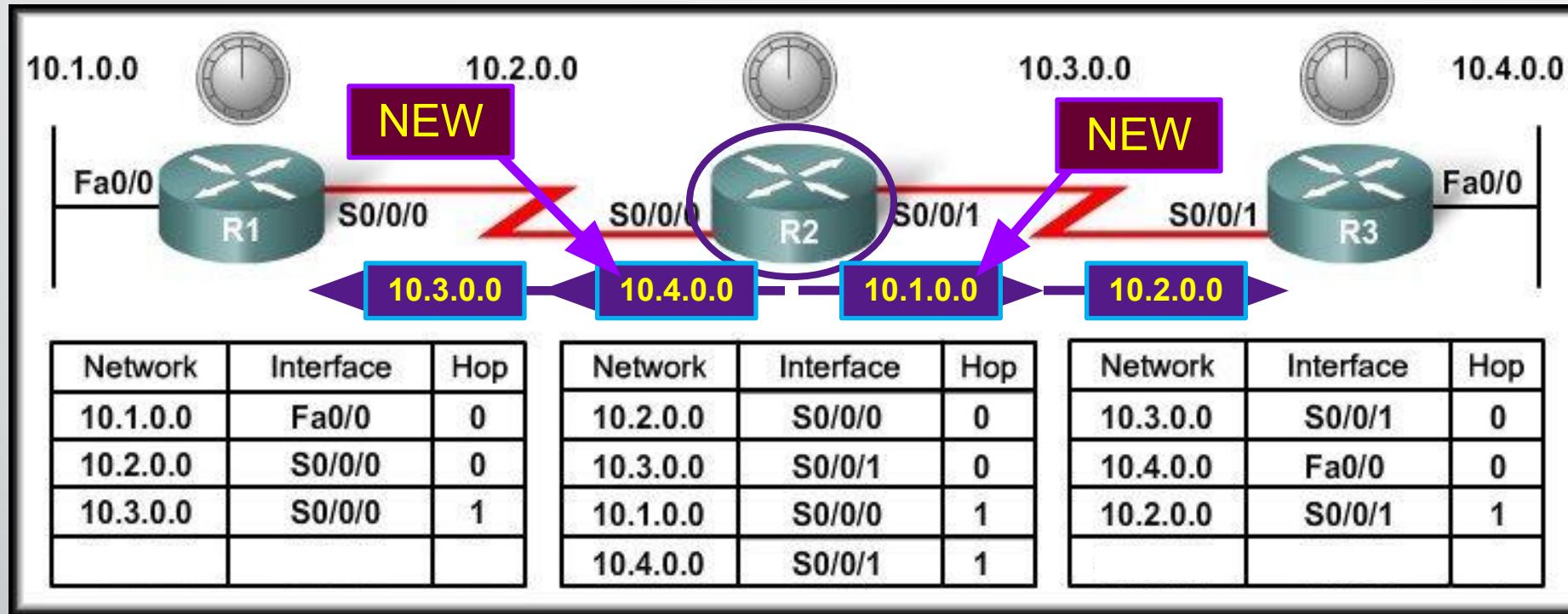
Initial Exchange of Routing Information



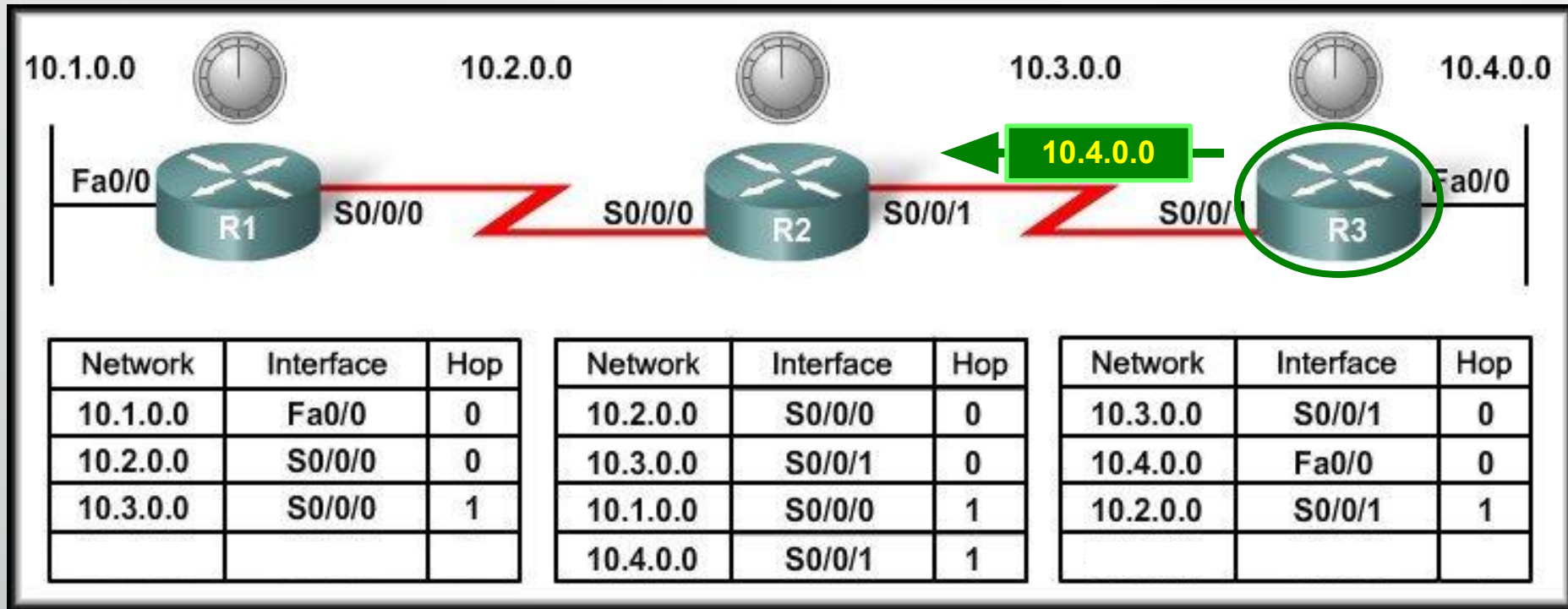
Next Exchange of Routing Information



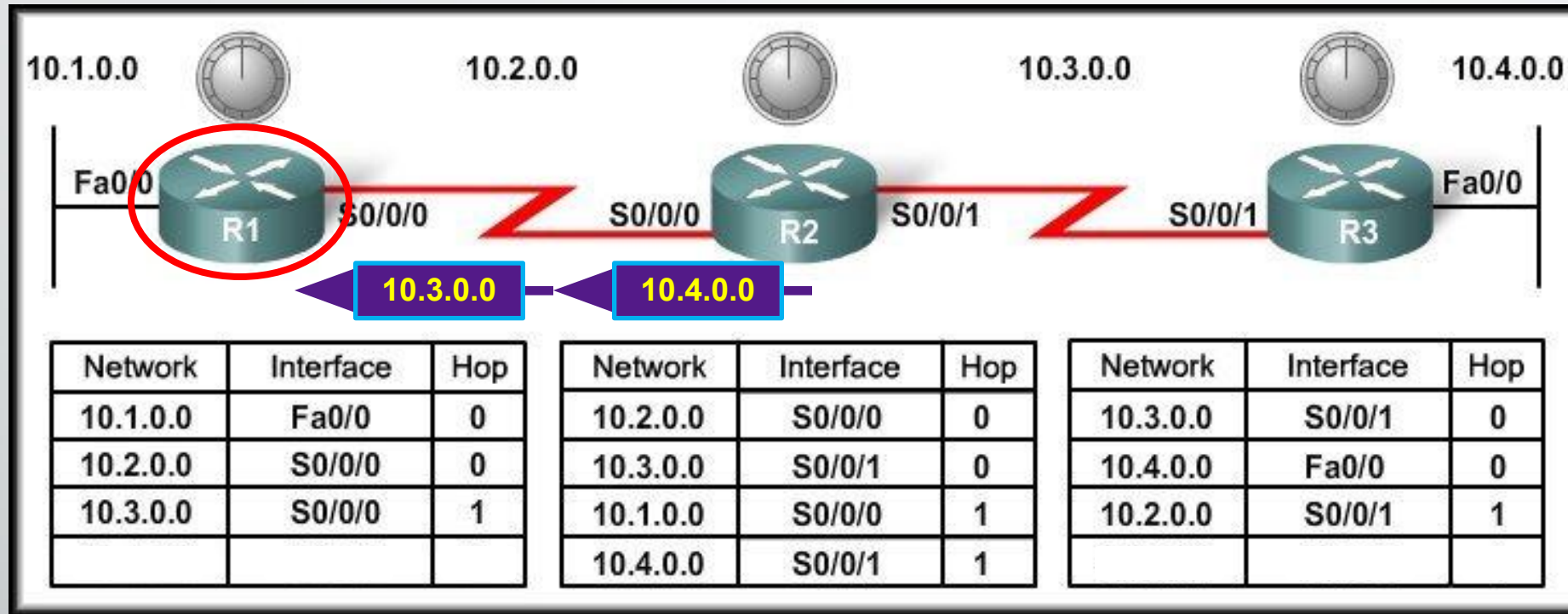
Next Exchange of Routing Information



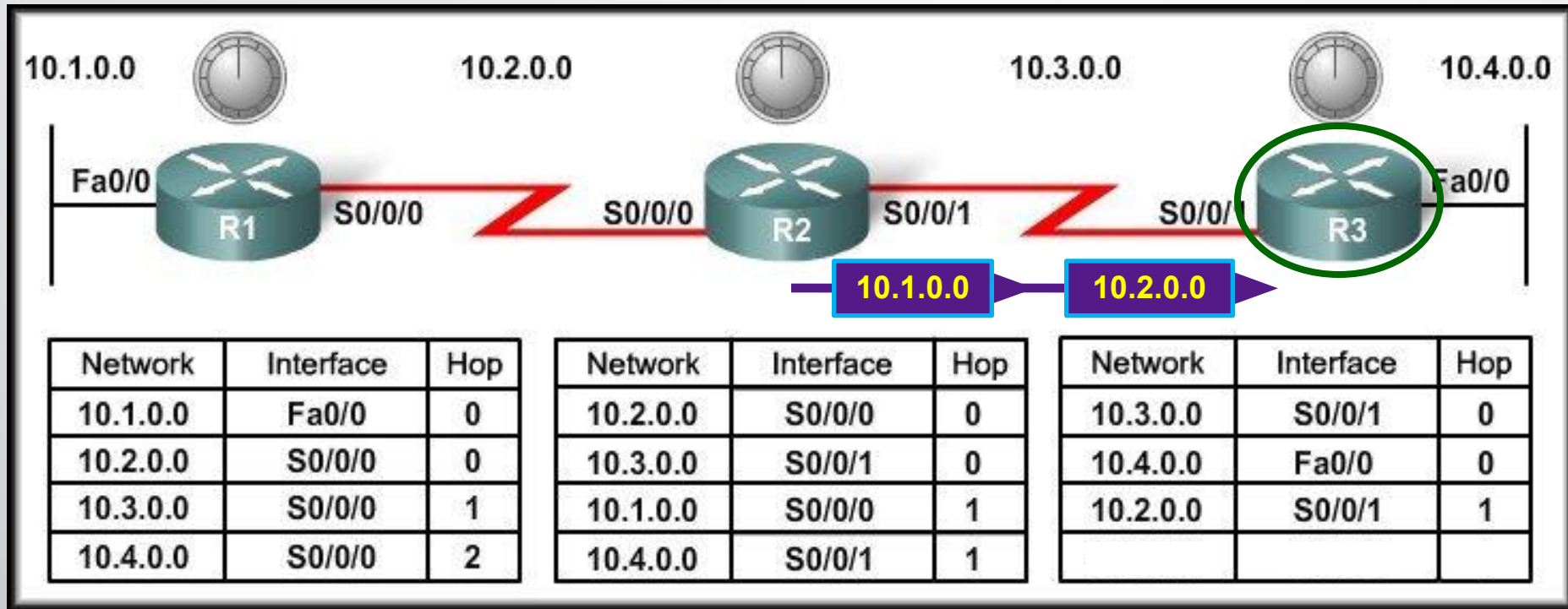
Next Exchange of Routing Information



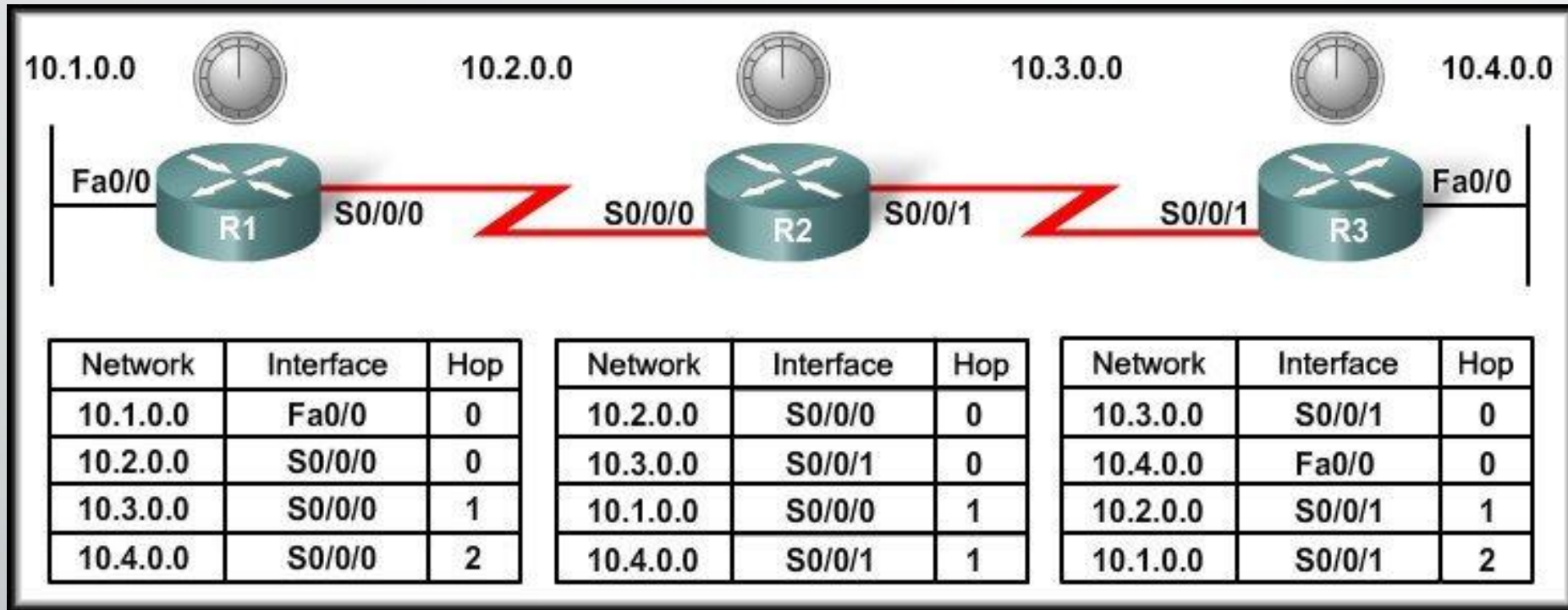
Next Exchange of Routing Information



Next Exchange of Routing Information



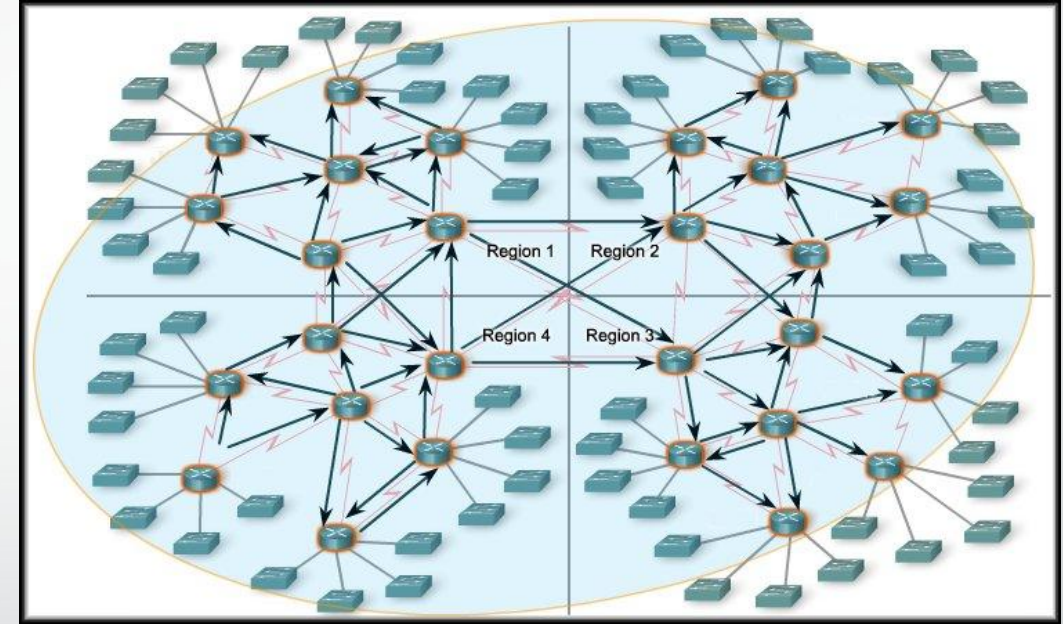
Next Exchange of Routing Information



- The network has **CONVERGED!**
 - All routers now know about all of the networks attached to all of their neighbouring routers.

Convergence

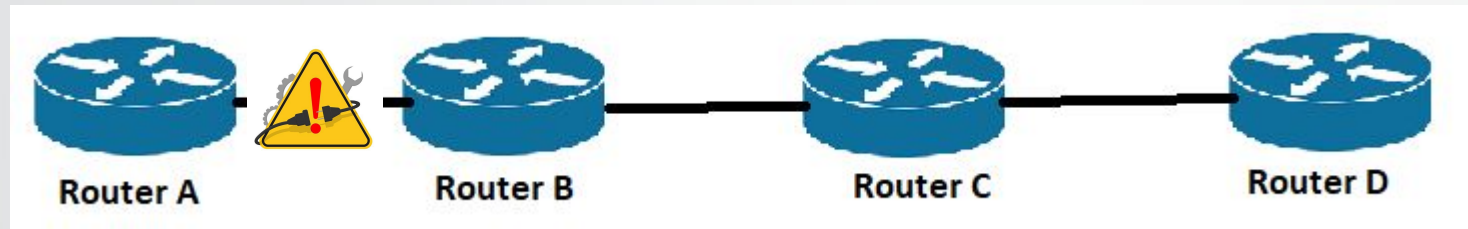
- **Convergence** is the time a network takes to reach a **stable routing state** after a change (e.g., link failure).
- In general, **larger networks** take **longer to converge**.
- Routing protocols are compared based on **how fast they spread routing updates**
→ this is called **speed of convergence**.
- A network **cannot forward traffic correctly** until convergence is complete.



Disadvantages

- Routers have **only local knowledge** (they know information only from neighbors).
- Routers **trust information received from neighbors**, even if it is outdated or incorrect.
- Routing updates are **slow to converge** after a link failure.
- Incorrect routing information can **spread from one router to another**.
- This can create **routing loops**.

Count to infinity Problem :: Routing Loops



	A	B	C	D
A	0, -	∞ , A	2, B	3, C
B	1, B	0, -	2, C	3, D
C	2, B	1, C	0, -	1, C
D	3, B	2, C	1, D	0, -



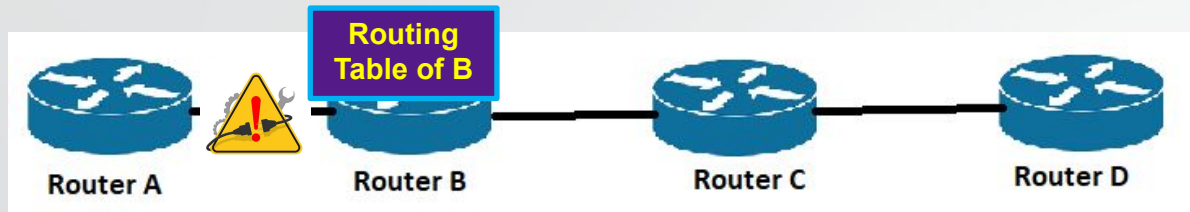
Count to infinity Problem :: Routing Loops



	B	C	D
Sum of Weight to A after link cut	∞ , A	2, B	3, C
Sum of Weight to A after 1 st updating	3, C	2, B	3, C

- After a specific amount of time, B receives C's routing table.
- Since C does not know that the link between A and B is down, it assumes a path to A with cost 2 (1 from C to B and 1 from B to A).
- B receives C's table and assumes there is an **alternate path to A via C**
- So it updates its distance to A from infinity to 3 (1 from B to C + 2 from C to A)
- Once again, routers exchange their tables.

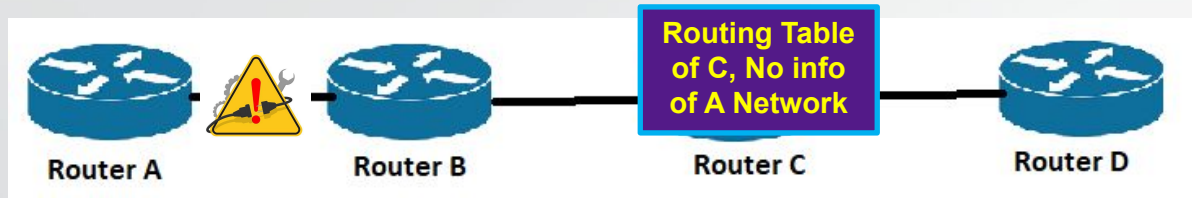
Count to infinity Problem :: Routing Loops



	B	C	D
Sum of Weight to A after link cut	∞ , A	2, B	3, C
Sum of Weight to A after 1 st updating	3, C	2, B	3, C

- When C receives B's routing table, it sees that B has changed the weight of its link to A from 1 to 3.
- So C updates its table and changes the weight of the link to A to 4 (1 for C to B, and 3 for B to A, as B said).
- This process loops until all nodes find out that the weight of link to A is infinity.

Solution One: Split Horizon Rule



- One way to solve this problem is that a router does not advertise a route back out the interface on which it was learned.
- Or “Poison reverse”: advertise that route back to the neighbor with an infinite metric.
- In this case, C shouldn't send any information to B about A, because B is the only way to A.



THE END